



The updated code implements a Flight Airline Management System using Python, MySQL, and external APIs (AeroDataBox and OpenWeatherMap). The system allows users to register/login, view/book/cancel flights, and admins to create/delete/delay flights with route optimization. Below is a file-by-file, line-by-line explanation, followed by the execution flow when the program runs.

File: auth.py

This file handles user authentication, including database connection, user registration, and login functionality.

Imports

- Line 1: Imports mysql.connector (aliased as db) to interact with the MySQL database.
- Line 2: Imports bcrypt for secure password hashing.

Function: get_connection

- Line 4: Defines a function to create a MySQL database connection.
- Line 5: Returns a connection object using mysql.connector.connect with hardcoded credentials (host="localhost", user="root", passwd="mysql", database="flights").
 - Purpose: Establishes a connection to the flights database for other functions to use.

Function: register

- Line 7: Defines a function to register a new user with a username, password, and optional role (defaults to "user").
- Line 8: Calls get_connection() to get a database connection.
- Line 9: Creates a cursor object to execute SQL queries.
- Line 10: Hashes the password using bcrypt for security.
- Line 11: Executes an INSERT query to save the username, hashed password, and role into the users table.
- Line 12: Commits the transaction.
- Line 13: Closes the connection and prints a success message.

Function: login

- Line 15: Defines a function to authenticate a user with a username and password.
- Line 16: Calls get_connection() to get a database connection.
- Line 17: Creates a cursor object and executes a SELECT query to fetch user details.
- Line 18: Closes the connection.
- Line 19-22: Checks the password using bcrypt and returns user details if valid, otherwise returns None with an error message.

File: admin_actions.py

This file contains admin-specific functions for flight management.

Function: create_flight

- Line 4: Imports necessary modules and functions, including flight_optimizer.
- Line 6: Defines a function to create and save an optimized flight with origin, destination, hubs, departure time, and passenger count.
- Line 7: Calls optimize_with_connection to get the best flight route.
- Line 8-15: Handles direct or connection flights, calculating arrival times and layovers.
- Line 16-18: Saves each flight segment to the database using save_flight.

Function: save_flight

- Line 20: Defines a function to save flight details to the database.
- Line 21-28: Establishes a connection, calculates arrival time, and executes an INSERT query with flight details.
- Line 29-31: Commits the transaction, closes the connection, and prints success/error messages.

Function: delete_flight

- Line 33: Defines a function to delete a flight by ID.
- Line 34-39: Establishes a connection, executes a DELETE query, commits, and prints success/error messages.

Function: delay_flight

- Line 41: Defines a function to delay a flight by minutes.
- Line 42-48: Establishes a connection, executes an UPDATE query to adjust departure and arrival times, commits, and prints success/error messages.

File: flight_optimizer.py

This file optimizes flight paths using external APIs.

Utility Functions

- Line 10-19: Defines get_airport_coords to fetch airport coordinates from AeroDataBox.

- Line 21-28: Defines haversine_distance to calculate great-circle distance.
- Line 30-37: Defines calculate_bearing for initial heading.
- Line 39-46: Defines midpoint_coords for great-circle midpoint.
- Line 48-56: Defines get_wind_data to fetch wind speed and direction from OpenWeatherMap.
- Line 58-71: Defines flight_traffic_on_route to count flights on a route.

Core Optimization

- Line 74-109: Defines optimize_flight_path to calculate the best plane based on fuel and time, considering wind and traffic.
- Line 111-124: Defines optimize_with_connection to optimize routes with hubs.

Visualization

- Line 126-141: Defines plot_route to visualize flight routes on a world map (not executed in this context).

File: main.py

This file serves as the program entry point.

- Line 1-3: Imports functions from other modules.
- Line 5-24: Defines user_menu with options to view flights, book, cancel, and view bookings.
- Line 26-43: Defines admin_menu with options to create, delete, and delay flights.
- Line 45-52: Handles login and directs to user or admin menu based on role.

File: user_actions.py

This file contains user-specific functions.

Function: view_active_flights

- Line 4: Defines a function to view active flights.

- Line 5-14: Fetches active flights and plots their routes using plot_route.
- Line 15: Returns the flight list.

Function: book_flight

- Line 17: Defines a function to book a flight.
- Line 18-21: Inserts a booking into the database and prints a success message.

Function: cancel_booking

- Line 23: Defines a function to cancel a booking.
- Line 24-27: Deletes a booking from the database and prints a success message.

Function: view_my_bookings

- Line 29-34: Defines a function to view a user's bookings with flight details.

Execution Flow

1. Program starts in main.py, prompting for login credentials.
2. auth.login authenticates the user and determines their role.
3. If admin, admin_menu is called; if user, user_menu is called.
4. User/admin actions trigger respective functions, interacting with the database and APIs as needed.
5. Flight optimization and visualization occur during flight creation/viewing.

This system efficiently manages flight operations with secure authentication and optimized routing.